

RawRepresentable

A type that can be converted to and from an associated raw value.

RawRepresentable

protocol RawRepresentable

A type that can be converted to and from an associated raw value.

Conforming Types

CodingUserInfoKey, OptionSet

Associated Types

associatedtype RawValue

Every distinct value of the conforming type has a corresponding unique value of the RawValue type, but there may be values of the RawValue type that don't have a corresponding value of the conforming type.

RawRepresentable

In Swift, we often need to work with values that can be represented in a simpler form, like converting an enum case to a string or an integer. This is where `RawRepresentable` comes into play.

It's a protocol that helps you bridge the gap between these complex values and simpler, more primitive types.

The `RawRepresentable` protocol is seen mainly in two categories of types: enumerations with raw value types and option sets.

Enumerations with Raw Values

For any enumeration with a string, integer, or floating-point raw type, the Swift compiler automatically adds `RawRepresentable` conformance.

Example: Assigning Tasks

```
enum TaskPriority: String {  
    case high = "High"  
    case medium = "Medium"  
    case low = "Low"  
}
```

In this case, `TaskPriority` conforms to `RawRepresentable`, which means each enum case is associated with a raw value (in this case, a `String`). This makes it easy to convert between the enum and its raw value.

Enumerations with Raw Values

In this case, `TaskPriority` conforms to `RawRepresentable`, which means each enum case is associated with a raw value (in this case, a `String`). This makes it easy to convert between the enum and its raw value:

```
let priority = TaskPriority.high
print(priority.rawValue) // Outputs: "High"

if let mediumPriority = TaskPriority(rawValue: "Medium") {
    print("Successfully created a medium priority task!")
}
```

priority.rawValue: This property is provided by the protocol. It allows you to "extract" the underlying string. This is useful for saving data to a database or displaying a label in a UI.

Before RawRepresentable

Before Swift introduced `RawRepresentable`, developers would manually convert enums to their corresponding values. This often meant writing a lot of repetitive code:

```
enum TaskPriority {  
    case high, medium, low  
}  
  
func priorityToString(priority: TaskPriority)  
-> String {  
    switch priority {  
    case .high:  
        return "High"  
    case .medium:  
        return "Medium"  
    case .low:  
        return "Low"  
    }  
}
```

```
func stringToPriority(string: String) ->  
TaskPriority? {  
    switch string {  
    case "High":  
        return .high  
    case "Medium":  
        return .medium  
    case "Low":  
        return .low  
    default:  
        return nil  
    }  
}
```

Swift Optionset

A type that presents a mathematical set interface to a bit set.

RawRepresentable

protocol OptionSet

A type that presents a mathematical set interface to a bit set.

Inheritance

RawRepresentable, SetAlgebra

Associated Types

associatedtype Element

To inherit all the default implementations from the OptionSet protocol, the Element type must be Self, the default.

Optionset

You use the `OptionSet` protocol to represent bitset types, where individual bits represent members of a set.

Adopting this protocol in your custom types lets you perform set-related operations such as membership tests, unions, and intersections on those types.

Optionsets can be thought of as an extension of enums that **allow multiple cases to be true simultaneously**

As Swift developers, various tools are at our disposal to handle different choice modelling scenarios effectively.

Two commonly used constructs for modeling options and choices are **enums** and **optionsets**. While enums are widely popular and powerful, there are cases where optionsets offer us more flexibility.

Optionset

What is an **OptionSet**?

An optionset is a data type that represents a set of distinct options or flags using bit-wise operations.

For simplicity, it wouldn't be wrong if you thought of Optionsets as essentially enums that allow multiple cases to be true at once.

They provide us the ability to model scenarios where we need multiple options to be combinable or where multiple cases can be true simultaneously.

Working with Option Set:

To define an optionset in Swift, we create a new struct and adopt the OptionSet protocol.

This protocol requires the conforming type to be a **RawRepresentable enumeration**, where the raw value should be an integer data type (Int, UInt, Int8, UInt8, etc.).

Optionset

Example

```
// Define an Option Set for Pizza Toppings
struct PizzaToppings: OptionSet { let
rawValue: Int

    static let cheese = PizzaToppings(rawValue: 1 << 0)
    static let pepperoni = PizzaToppings(rawValue: 1 << 1)
    static let mushrooms = PizzaToppings(rawValue: 1 << 2)
    static let onions = PizzaToppings(rawValue: 1 << 3)
    static let bellPeppers = PizzaToppings(rawValue: 1 << 4)
// Add more toppings as needed }
```

Optionset

Example

```
// Define an Option Set for Pizza Toppings
struct PizzaToppings: OptionSet {
    let rawValue: Int
    static let cheese = PizzaToppings(rawValue: 1 << 0)
    static let pepperoni = PizzaToppings(rawValue: 1 << 1)
    static let mushrooms = PizzaToppings(rawValue: 1 << 2)
    static let onions = PizzaToppings(rawValue: 1 << 3)
    static let bellPeppers = PizzaToppings(rawValue: 1 << 4)
    // Add more toppings as needed
}

// Define an Enum for Pizza Size
enum PizzaSize: String {
    case small
    case medium
    case large
}

// Define a Pizza struct
struct Pizza {
    let size: PizzaSize
    var toppings: PizzaToppings
}
```

```
// Create a pizza
var myPizza = Pizza(size: .medium, toppings: [.cheese, .mushrooms])

// Add more toppings
myPizza.toppings.insert(.pepperoni)

// Check if the pizza has onions
if myPizza.toppings.contains(.onions) {
    print("This pizza has onions.")
} else {
    print("This pizza does not have onions.")
}

// Display the pizza details
print("Size: \(myPizza.size.rawValue.capitalized)")
print("Toppings: \(myPizza.toppings)")
```

Optionset

Exa mpl

In the given code We created a Pizza struct that contains the size and toppings properties just like we wanted. We then created a medium sized pizza with cheese and mushroom toppings. Later, we add pepperoni as an additional topping to the pizza.

Finally, we demonstrate how to check if the pizza has onions using the **contains()** method of the option set, and then display the pizza details, including the size and toppings.

Although we can initialize an Optionset with an array literal syntax like this `pizzaToppings = []`. The **Optionset** protocol is actually **more like a Set than an array**, therefore they can be manipulated using set operations, such as **union**, **intersection**, and **subtraction**, making it easy to combine, check, and modify your set of options.

Optionset

Example

Let's play with the various methods and operators we can apply on them:

```
// Define a set of all available toppingslet allToppings: PizzaToppings = [.cheese, .pepperoni, .mushrooms, .onions, .bellPeppers]
```

```
// Get the toppings common to the pizza and the available toppingslet commonToppings = myPizza.intersection(allToppings)
```

```
// Remove a topping from the pizzamyPizza.subtract(.pepperoni)
```

```
// Get the toppings that are not present in the pizza but availablelet missingToppings = allToppings.subtracting(myPizza)
```

```
// Display the pizza detailsprint("Selected Toppings: \(myPizza)")print("Common Toppings: \(commonToppings)")print("Missing Toppings: \(missingToppings)")
```